

SD Basic - Syntax

Every statement and function, alphabetically, with its syntax and nothing else.

A lookup card, and nothing else. If you know what you want and have forgotten how to spell it, it is here. If you want to know what it *does*, the other seventeen documents in this set are where that lives.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part. SD folds case, so any of this may be written in either case.

This page is generated, and it is checked for completeness rather than proof-read for it. The roster comes from BCOMP's own tables — the same extraction the rest of this set uses — and tools/mksyntax.py refuses to write the page if a single name that belongs on it has no line. Argument counts for functions are read out of BCOMP's dispatch table, which is positional against the name list and carries each name in a comment; the script asserts the two agree before it uses either. The shapes that a count cannot express — every statement, and about twenty functions — come from documents 01 to 17, where each one is described in full.

What is not here, and where it went. Everything on this card is something an application may use. Names that an ordinary program **cannot** compile are in the administrator documentation, under *SD Basic - Restricted Commands*: the restricted statements, the internal-only functions, and the one name that is in the compiler's table with nothing behind it. **If you are looking for something and it is not here, that is where to look before concluding it does not exist.**

ONE THING IS MARKED, AND IT IS THE ONE THAT WASTES TIME.

(clause word) not a statement — a word that belongs inside another one's syntax

A

abort	abort {message}
abs	abs(a)
abss	abss(a)
accept.socket.connection	accept.socket.connection(a, b)
acos	acos(a)

<code>alpha</code>	<code>alpha(<i>a</i>)</code>
<code>ands</code>	<code>ands(<i>a</i>, <i>b</i>)</code>
<code>append</code>	clause of <code>openseq</code> , and of <code>edit</code> (<i>clause word</i>)
<code>arg</code>	<code>arg(<i>a</i>)</code>
<code>arg.count</code>	<code>arg.count()</code>
<code>ascii</code>	<code>ascii(<i>a</i>)</code>
<code>asin</code>	<code>asin(<i>a</i>)</code>
<code>assigned</code>	<code>assigned(<i>a</i>)</code>
<code>atan</code>	<code>atan(<i>a</i>)</code>

B

<code>before</code>	clause of <code>ins</code> : <code>ins value before arr< <i>f</i> ></code> (<i>clause word</i>)
<code>begin</code>	<code>begin case ... end case</code> · <code>begin transaction ... end transaction</code>
<code>bindkey</code>	<code>bindkey(<i>a</i>, <i>b</i>)</code>
<code>bitand</code>	<code>bitand(<i>a</i>, <i>b</i>)</code>
<code>bitnot</code>	<code>bitnot(<i>a</i>)</code>
<code>bitor</code>	<code>bitor(<i>a</i>, <i>b</i>)</code>
<code>bitreset</code>	<code>bitreset(<i>a</i>, <i>b</i>)</code>
<code>bitset</code>	<code>bitset(<i>a</i>, <i>b</i>)</code>
<code>bittest</code>	<code>bittest(<i>a</i>, <i>b</i>)</code>
<code>bitxor</code>	<code>bitxor(<i>a</i>, <i>b</i>)</code>
<code>break</code>	<code>break on</code> · <code>break off</code> · <code>break <i>n</i></code>
<code>by</code>	clause of <code>locate</code> and <code>sselect</code> — <code>by 'AL', 'AR', 'DL', 'DR'</code> (<i>clause word</i>)

C

<code>call</code>	<code>call name (arg, ...) · call @ var (arg, ...)</code>
-------------------	---

<code>capturing</code>	clause of <code>execute</code> and <code>os.execute</code> — take the output (<i>clause word</i>)
------------------------	--

<code>case</code>	<code>case condition</code> , inside <code>begin case</code> ... <code>end case</code>
-------------------	--

<code>catalogued</code>	<code>catalogued(a)</code>
-------------------------	------------------------------

<code>cats</code>	<code>cats(a, b)</code>
-------------------	---------------------------

<code>ccall</code>	<code>ccall(a, b)</code>
--------------------	----------------------------

<code>chain</code>	<code>chain command</code>
--------------------	----------------------------

<code>change</code>	<code>change(string, from, to {, occurrences {, start}})</code>
---------------------	---

<code>char</code>	<code>char(a)</code>
-------------------	------------------------

<code>checksum</code>	<code>checksum(a)</code>
-----------------------	----------------------------

<code>chgphant</code>	<code>chgphant()</code>
-----------------------	-------------------------

<code>class</code>	<code>class name</code>
--------------------	-------------------------

<code>clear</code>	<code>clear</code> — zeroes every variable
--------------------	--

<code>clearcommon</code>	<code>clearcommon {name}</code>
--------------------------	---------------------------------

<code>cleardata</code>	<code>cleardata</code>
------------------------	------------------------

<code>clearfile</code>	<code>clearfile file.var { on error ... }</code>
------------------------	--

<code>clearinput</code>	<code>clearinput</code>
-------------------------	-------------------------

<code>clearselect</code>	<code>clearselect {list.no}</code>
--------------------------	------------------------------------

<code>close</code>	<code>close file.var { on error ... }</code>
--------------------	--

<code>close.socket</code>	<code>close.socket socket</code>
---------------------------	----------------------------------

<code>closeseq</code>	<code>closeseq file.var</code>
-----------------------	--------------------------------

<code>col1</code>	<code>col1()</code>
-------------------	---------------------

<code>col2</code>	<code>col2()</code>
-------------------	---------------------

<code>com</code>	<code>com</code> / <i>name</i> / <i>var</i> , ... — synonym of <code>common</code>
<code>commit</code>	<code>commit</code> , inside <code>begin transaction</code> ... <code>end transaction</code>
<code>common</code>	<code>common</code> / <i>name</i> / <i>var</i> , <i>matrix</i> (<i>n</i>), ...
<code>compare</code>	<code>compare</code> (<i>a</i> , <i>b</i> {, <i>justification</i> })
<code>config</code>	<code>config</code> (<i>a</i>)
<code>continue</code>	<code>continue</code> — next iteration of the enclosing <code>loop</code> or <code>for</code>
<code>convert</code>	<code>convert</code> (<i>a</i> , <i>b</i> , <i>c</i>)
<code>cos</code>	<code>cos</code> (<i>a</i>)
<code>count</code>	<code>count</code> (<i>a</i> , <i>b</i>)
<code>counts</code>	<code>counts</code> (<i>a</i> , <i>b</i>)
<code>create</code>	<code>create</code> <i>file.var</i> <code>then</code> ... <code>else</code> ...
<code>create.file</code>	<code>create.file</code> <i>name</i> { <i>directory</i> } { <code>on error</code> ... }
<code>create.server.socket</code>	<code>create.server.socket</code> (<i>addr</i> , <i>port</i> {, <i>flags</i> })
<code>crop</code>	<code>crop</code> (<i>a</i>)
<code>crt</code>	<code>crt</code> <i>expr</i> { <code>:</code> ... } { <code>:</code> }
<code>csvdq</code>	<code>csvdq</code> (<i>line</i> {, <i>delimiter</i> })
<code>current.level</code>	clause of <code>enter</code> — run at the current command level (<i>clause word</i>)

D

<code>data</code>	<code>data</code> <i>expr</i> , ...
<code>date</code>	<code>date</code> ()
<code>dcount</code>	<code>dcount</code> (<i>a</i> , <i>b</i>)
<code>debug</code>	<code>debug</code> — needs <code>basic</code> <i>file record</i> <code>debugging</code>

deffun	<code>deffun</code> <i>name</i> (<i>arg</i> , ...) { <code>calling</code> "cat.name" } { <code>local</code> }
del	<code>del</code> <i>arr</i> < <i>f</i> {, <i>v</i> {, <i>sv</i> }} >
delete	<code>delete</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... } · <code>delete</code> (<i>arr</i> , <i>f</i> {, <i>v</i> {, <i>sv</i> }})
deletelist	<code>deletelist</code> <i>name</i> <code>then</code> ... <code>else</code> ...
deleteseq	<code>deleteseq</code> <i>path</i> { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
deleteu	<code>deleteu</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... }
dim	<code>dim</code> <i>name</i> (<i>rows</i> {, <i>cols</i> }) , ...
dimension	<code>dimension</code> <i>name</i> (<i>rows</i> {, <i>cols</i> }) , ... — synonym of <code>dim</code>
dir	<code>dir</code> (<i>a</i>)
disinherit	<code>disinherit</code> <i>object</i>
display	<code>display</code> <i>expr</i> { : ... } { : }
div	<code>div</code> (<i>a</i> , <i>b</i>)
do	<code>loop</code> ... <code>do</code> ... <code>repeat</code>
downcase	<code>downcase</code> (<i>a</i>)
dparse	<code>dparse</code> <i>string</i> , <i>delimiter</i> , <i>var</i> , ...
dparse.csv	<code>dparse.csv</code> <i>line</i> , <i>delimiter</i> , <i>var</i> , ...
dquote	<code>dquote</code> (<i>a</i>)
dtx	<code>dtx</code> (<i>n</i> {, <i>digits</i> })

E

ebcdic	<code>ebcdic</code> (<i>a</i>)
echo	<code>echo on</code> · <code>echo off</code> · <code>echo</code> <i>expr</i>
edit	clause of <code>keyedit</code> , and the <code>edit</code> verb at the command prompt (clause word)

else	clause of if , and of every statement that has a then
end	end — closes a program, a then , an else , a case , a transaction
enter	enter <i>name</i> · enter @ <i>var</i>
env	env (<i>a</i>)
eqs	eqs (<i>a</i> , <i>b</i>)
equ	equ <i>name</i> to <i>value</i> — synonym of equate
equate	equate <i>name</i> to <i>value</i> {, <i>name</i> to <i>value</i> } ...
execute	execute <i>cmd</i> { capturing <i>v</i> } { returning <i>v</i> } { passlist { <i>l</i> } } { rtnlist { <i>v</i> } }
exit	exit — leave the enclosing loop or for
exp	exp (<i>a</i>)
extract	extract (<i>arr</i> , <i>f</i> {, <i>v</i> {, <i>sv</i> } })

F

field	field (<i>string</i> , <i>delimiter</i> , <i>occurrence</i> {, <i>count</i> })
fields	fields (<i>arr</i> , <i>delimiter</i> , <i>occurrence</i> {, <i>count</i> })
fieldstore	fieldstore (<i>a</i> , <i>b</i> , <i>c</i> , <i>d</i> , <i>e</i>)
file	file <i>name</i> — names a file at compile time
fileinfo	fileinfo (<i>file.var</i> , <i>key</i>)
filelock	filelock <i>file.var</i> { locked ... } { on error ... } — no then / else
fileunlock	fileunlock <i>file.var</i> { on error ... }
findstr	findstr <i>x</i> in <i>arr</i> {, <i>occ</i> } setting <i>f</i> {, <i>v</i> {, <i>sv</i> } } then ... else ...
flush	flush <i>file.var</i> then ... else ...
fmt	fmt (<i>a</i> , <i>b</i>)

fmts	<code>fmts(<i>a</i>, <i>b</i>)</code>
fold	<code>fold(<i>string</i>, <i>width</i> {, <i>delimiter</i>})</code>
folds	<code>folds(<i>arr</i>, <i>width</i> {, <i>delimiter</i>})</code>
footing	<code>footing</code> <i>expr</i>
for	<code>for</code> <i>var</i> = <i>start</i> <code>to</code> <i>end</i> { <code>step</code> <i>n</i> } ... <code>next</code> <i>var</i>
formlist	<code>formlist</code> <i>arr</i> {, <i>delimiter</i> } { <code>to</code> <i>list.no</i> }
from	clause of <code>read</code> , <code>readnext</code> , <code>matbuild</code> , <code>matparse</code> , <code>remove</code> , <code>readcsv</code> (clause word)
function	<code>function</code> <i>name</i> (<i>arg</i> , ...)

G

ges	<code>ges(<i>a</i>, <i>b</i>)</code>
get	<code>get</code> <i>var</i> <code>from</code> <i>object</i>
get.messages	<code>get.messages()</code>
get.port.params	<code>get.port.params(<i>a</i>)</code>
getlist	<code>getlist</code> <i>name</i> { <code>to</code> <i>list.no</i> } <code>then</code> ... <code>else</code> — the <code>then</code> / <code>else</code> is not optional
getnls	<code>getnls(<i>a</i>)</code>
getpu	<code>getpu(<i>a</i>, <i>b</i>)</code>
getrem	<code>getrem(<i>a</i>)</code>
go	<code>go</code> <i>label</i> — synonym of <code>goto</code>
gosub	<code>gosub</code> <i>label</i> {, <i>arg</i> , ...}
goto	<code>goto</code> <i>label</i>
gts	<code>gts(<i>a</i>, <i>b</i>)</code>

H

<code>heading</code>	<code>heading</code> <i>expr</i>
<code>hidden</code>	clause of <code>input</code> — do not echo what is typed (<i>clause word</i>)
<code>hush</code>	<code>hush on</code> · <code>hush off</code> · <code>hush</code> <i>expr</i>

I

<code>iconv</code>	<code>iconv(</code> <i>a</i> <code>,</code> <i>b</i> <code>)</code>
<code>iconvs</code>	<code>iconvs(</code> <i>a</i> <code>,</code> <i>b</i> <code>)</code>
<code>idiv</code>	<code>idiv(</code> <i>a</i> <code>,</code> <i>b</i> <code>)</code>
<code>if</code>	<code>if</code> <i>condition</i> <code>then</code> ... <code>else</code> ...
<code>ifs</code>	<code>ifs(</code> <i>a</i> <code>,</code> <i>b</i> <code>,</code> <i>c</i> <code>)</code>
<code>in</code>	clause of <code>find</code> , <code>findstr</code> and <code>locate</code>
<code>include</code>	<code>\$include</code> <i>record</i> · <code>\$include</code> <i>file record</i>
<code>index</code>	<code>index(</code> <i>a</i> <code>,</code> <i>b</i> <code>,</code> <i>c</i> <code>)</code>
<code>indexs</code>	<code>indexs(</code> <i>a</i> <code>,</code> <i>b</i> <code>,</code> <i>c</i> <code>)</code>
<code>indices</code>	<code>indices(</code> <i>file.var</i> { <i>,</i> <i>index</i> } <code>)</code>
<code>inherit</code>	<code>inherit</code> <i>object</i>
<code>inmat</code>	<code>inmat()</code> · <code>inmat(</code> <i>matrix</i> <code>)</code>
<code>input</code>	<code>input</code> <i>var</i> { <i>length</i> } { <code>:</code> } { <code>_</code> } { <code>hidden</code> } { <code>overlay</code> } { <code>upcase</code> } { <code>waiting</code> <i>n</i> }
<code>inputblk</code>	<code>inputblk(</code> <i>a</i> <code>)</code>
<code>inputclear</code>	<code>inputclear</code>
<code>inputcsv</code>	<code>inputcsv</code> <i>var</i> , ...

inputerr	<code>inputerr</code> <i>message</i>
inputfield	<code>inputfield</code> <i>var, length {, mask}</i>
ins	<code>ins</code> <i>value</i> <code>before</code> <i>arr</i> <code>< f {, v {, sv}} ></code>
insert	<code>insert(</code> <i>arr, f {, v {, sv}}</i> , <i>value</i> <code>)</code>
int	<code>int(</code> <i>a</i> <code>)</code>
itype	<code>itype(</code> <i>compiled.i.type</i> <code>)</code>

K

keycode	<code>keycode(</code> <i>{timeout}</i> <code>)</code>
keyedit	<code>keyedit</code> <i>key, action</i>
keyexit	<code>keyexit</code> <i>key, action</i>
keyin	<code>keyin(</code> <i>{timeout}</i> <code>)</code>
keyinc	<code>keyinc(</code> <i>{timeout}</i> <code>)</code>
keyinr	<code>keyinr(</code> <i>{timeout}</i> <code>)</code>
keyready	<code>keyready()</code>
keytrap	<code>keytrap</code> <i>key, action</i>

L

len	<code>len(</code> <i>a</i> <code>)</code>
lens	<code>lens(</code> <i>a</i> <code>)</code>
les	<code>les(</code> <i>a, b</i> <code>)</code>
listindex	<code>listindex(</code> <i>a, b, c</i> <code>)</code>

<code>ln</code>	<code>ln(a)</code>
<code>local</code>	<code>local</code> <i>var</i> , ... — inside a <code>deffun</code> declared <code>local</code>
<code>locate</code>	<code>locate</code> <i>x</i> <code>in</code> <i>arr</i> <code>< f {, v} > { by order } setting pos then ... else ...</code>
<code>lock</code>	<code>lock</code> <i>n</i> <code>then</code> ... <code>else</code> ... — <i>with no else it retries for ever</i>
<code>locked</code>	clause of <code>readu</code> , <code>readl</code> , <code>readvu</code> , <code>readvl</code> , <code>matreadu</code> , <code>matreadl</code> , <code>filelock</code> , <code>recordlocku</code> , <code>recordlockl</code> (<i>clause word</i>)
<code>logmsg</code>	<code>logmsg</code> <i>text</i>
<code>loop</code>	<code>loop</code> ... { <code>while</code> <i>c</i> } { <code>until</code> <i>c</i> } ... <code>repeat</code>
<code>lower</code>	<code>lower(a)</code>
<code>lts</code>	<code>lts(a, b)</code>

M

<code>mark.mapping</code>	<code>mark.mapping</code> <i>file.var</i> , <i>state</i>
<code>mat</code>	<code>mat</code> <i>matrix</i> = <i>value</i> · <code>mat</code> <i>a</i> = <code>mat</code> <i>b</i>
<code>matbuild</code>	<code>matbuild</code> <i>var</i> <code>from</code> <i>matrix</i> { <i>start</i> {, <i>end</i> } } — <i>no using</i>
<code>matchfield</code>	<code>matchfield(a, b, c)</code>
<code>matparse</code>	<code>matparse</code> <i>matrix</i> <code>from</code> <i>string</i> {, <i>delimiter</i> }
<code>matread</code>	<code>matread</code> <i>matrix</i> <code>from</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
<code>matreadcsv</code>	<code>matreadcsv</code> <i>matrix</i> <code>from</code> <i>file.var</i> <code>then</code> ... <code>else</code> ...
<code>matreadl</code>	<code>matreadl</code> <i>matrix</i> <code>from</code> <i>file.var</i> , <i>id</i> { <code>locked</code> ... } <code>then</code> ... <code>else</code> ...
<code>matreadu</code>	<code>matreadu</code> <i>matrix</i> <code>from</code> <i>file.var</i> , <i>id</i> { <code>locked</code> ... } <code>then</code> ... <code>else</code> ...
<code>matwrite</code>	<code>matwrite</code> <i>matrix</i> <code>to</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... }
<code>matwriteu</code>	<code>matwriteu</code> <i>matrix</i> <code>to</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... }
<code>max</code>	<code>max(a, b)</code>

<code>maximum</code>	<code>maximum(<i>a</i>)</code>
----------------------	----------------------------------

<code>min</code>	<code>min(<i>a</i>, <i>b</i>)</code>
------------------	--

<code>minimum</code>	<code>minimum(<i>a</i>)</code>
----------------------	----------------------------------

<code>mod</code>	<code>mod(<i>a</i>, <i>b</i>)</code>
------------------	--

<code>mods</code>	<code>mods(<i>a</i>, <i>b</i>)</code>
-------------------	---

N

<code>nap</code>	<code>nap</code> <i>milliseconds</i>
------------------	--------------------------------------

<code>neg</code>	<code>neg(<i>a</i>)</code>
------------------	------------------------------

<code>negs</code>	<code>negs(<i>a</i>)</code>
-------------------	-------------------------------

<code>nes</code>	<code>nes(<i>a</i>, <i>b</i>)</code>
------------------	--

<code>next</code>	<code>next</code> <i>var</i> — closes a <code>for</code>
-------------------	--

<code>nobuf</code>	<code>nobuf</code> <i>file.var</i> <code>then</code> ... <code>else</code> ...
--------------------	--

<code>nocaseinvert</code>	clause of <code>input</code> — do not invert the case of typed letters (<i>clause word</i>)
---------------------------	--

<code>not</code>	<code>not(<i>a</i>)</code>
------------------	------------------------------

<code>nots</code>	<code>nots(<i>a</i>)</code>
-------------------	-------------------------------

<code>null</code>	<code>null</code> — does nothing; the empty statement
-------------------	---

<code>num</code>	<code>num(<i>a</i>)</code>
------------------	------------------------------

<code>nums</code>	<code>nums(<i>a</i>)</code>
-------------------	-------------------------------

O

<code>object</code>	<code>object(<i>class</i> { <i>arg</i>, ... })</code>
---------------------	---

objinfo	objinfo(<i>a</i> , <i>b</i>)
oconv	oconv(<i>a</i> , <i>b</i>)
oconvs	oconvs(<i>a</i> , <i>b</i>)
on	on <i>n</i> goto <i>label</i> , ... — clamps, it does not fall through
open	open { <i>dict</i> , } <i>name</i> to <i>file.var</i> {readonly} {on error ...} then ... else ...
open.socket	open.socket(<i>addr</i> , <i>port</i> , <i>flags</i> {, <i>context</i> })
openpath	openpath <i>path</i> to <i>file.var</i> {readonly} {on error ...} then ... else ...
openseq	openseq <i>path</i> to <i>file.var</i> {append} {overwrite} {readonly} then ... else ... — then = it existed, else = it was created
ors	ors(<i>a</i> , <i>b</i>)
os.error	os.error()
os.execute	os.execute <i>command</i> {capturing <i>var</i> }
outerjoin	outerjoin(<i>a</i> , <i>b</i> , <i>c</i>)
overlay	clause of input — type over what is already there (clause word)
overwrite	clause of openseq (clause word)

P

page	page { <i>n</i> }
panning	clause of input — pan a field wider than the screen (clause word)
passlist	clause of execute — hand the active select list to the command (clause word)
pause	pause { <i>seconds</i> }
perform	perform <i>command</i>
precision	precision <i>n</i>
print	print <i>expr</i> { <i>:</i> ...} { <i>:</i> } · print on <i>unit expr</i>

<code>printcsv</code>	<code>printcsv { on <i>unit</i> } value, ... { : }</code>
-----------------------	---

<code>printer</code>	<code>printer on</code> · <code>printer off</code> · <code>printer close</code> · <code>printer file on</code> <i>unit file, record</i>
----------------------	---

<code>printer.setting</code>	<code>printer.setting(<i>a</i>, <i>b</i>, <i>c</i>)</code>
------------------------------	--

<code>printerr</code>	<code>printerr</code> <i>message</i>
-----------------------	--------------------------------------

<code>private</code>	<code>private</code> <i>var</i> , ...
----------------------	---------------------------------------

<code>procread</code>	<code>procread</code> <i>var</i> <code>then</code> ... <code>else</code> ...
-----------------------	--

<code>procwrite</code>	<code>procwrite</code> <i>var</i>
------------------------	-----------------------------------

<code>program</code>	<code>program</code> <i>name</i>
----------------------	----------------------------------

<code>public</code>	<code>public</code> <i>var</i> , ...
---------------------	--------------------------------------

<code>pwr</code>	<code>pwr(<i>a</i>, <i>b</i>)</code>
------------------	--

Q

<code>quote</code>	<code>quote(<i>a</i>)</code>
--------------------	--------------------------------

R

<code>raise</code>	<code>raise(<i>a</i>)</code>
--------------------	--------------------------------

<code>randomize</code>	<code>randomize</code> · <code>randomize(<i>seed</i>)</code>
------------------------	--

<code>rdiv</code>	<code>rdiv(<i>a</i>, <i>b</i>)</code>
-------------------	---

<code>read</code>	<code>read</code> <i>var</i> <code>from</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
-------------------	--

<code>read.socket</code>	<code>read.socket(<i>a</i>, <i>b</i>, <i>c</i>, <i>d</i>)</code>
--------------------------	--

<code>readblk</code>	<code>readblk</code> <i>var</i> <code>from</code> <i>file.var</i> , <i>length</i> <code>then</code> ... <code>else</code> ...
----------------------	---

<code>readcsv</code>	<code>readcsv</code> <code>from</code> <i>file.var</i> <code>to</code> <i>var</i> , ... <code>then</code> ... <code>else</code> ...
----------------------	---

readl	<code>readl var from file.var, id {locked ...} {on error ...} then ... else ...</code>
readlist	<code>readlist var {from list.no} then ... else ...</code>
readnext	<code>readnext id {, vm} {from list.no} {on error ...} then ... else ...</code>
readonly	clause of <code>open</code> , <code>openpath</code> and <code>openseq</code> (clause word)
readseq	<code>readseq var from file.var {on error ...} then ... else ...</code> — <code>else</code> is end of file
readu	<code>readu var from file.var, id {locked ...} {on error ...} then ... else ...</code>
readv	<code>readv var from file.var, id, field {on error ...} then ... else ...</code>
readv1	<code>readv1 var from file.var, id, field {locked ...} then ... else ...</code>
readvu	<code>readvu var from file.var, id, field {locked ...} then ... else ...</code>
recordlocked	<code>recordlocked(file.var, id)</code> — -3 -2 -1 0 1 2 3
recordlock1	<code>recordlock1 file.var, id {locked ...} {on error ...}</code>
recordlocku	<code>recordlocku file.var, id {locked ...} {on error ...}</code>
release	<code>release {file.var {, id}} {on error ...}</code> — bare <code>release</code> releases everything
rem	<code>rem(a, b)</code>
remark	<code>remark text</code> — synonym of <code>rem</code>
remove	<code>remove value from arr setting delimiter.code</code>
remove.break.handler	<code>remove.break.handler</code>
repeat	<code>repeat</code> — closes a <code>loop</code>
replace	<code>replace(arr, f {, v {, sv}}, value)</code>
restore.screen	<code>restore.screen image, restore.cursor</code>
return	<code>return {to label} · return(value)</code> from a function
returning	clause of <code>execute</code> — take the command's error text (clause word)
reuse	<code>reuse(a)</code>
rnd	<code>rnd(a)</code>

<code>rollback</code>	<code>rollback</code> , inside <code>begin transaction</code> ... <code>end transaction</code>
<code>rqm</code>	<code>rqm {seconds}</code>
<code>rtnlist</code>	clause of <code>execute</code> — take the list the command left (clause word)
<code>rtrans</code>	<code>rtrans(file.var, id, field, action)</code>

S

<code>save.screen</code>	<code>save.screen(a, b, c, d)</code>
<code>savelist</code>	<code>savelist name { from list.no } then ... else</code> — the then / else is not optional
<code>sddecrypt</code>	<code>sddecrypt(a, b, c)</code>
<code>sdencrypt</code>	<code>sdencrypt(a, b, c)</code>
<code>seek</code>	<code>seek file.var, offset {, relative.to} then ... else ...</code>
<code>select</code>	<code>select {file.var} { to list.no }</code>
<code>selecte</code>	<code>selecte { to var }</code>
<code>selectindex</code>	<code>selectindex index {, value} from file.var { to list.no }</code>
<code>selectinfo</code>	<code>selectinfo(a, b)</code>
<code>selectleft</code>	<code>selectleft index from file.var setting var to list.no</code> — no then / else
<code>selectn</code>	<code>selectn {file.var} to list.no</code>
<code>selectright</code>	<code>selectright index from file.var setting var to list.no</code> — no then / else
<code>selectv</code>	<code>selectv {file.var} to var</code> — read it back with <code>readnext ... from</code>
<code>sendmail</code>	<code>sendmail to, from, subject, body {, attachments} then ... else ...</code>
<code>sentence</code>	<code>sentence()</code> — the same value as <code>@sentence</code>
<code>seq</code>	<code>seq(a)</code>
<code>server.addr</code>	<code>server.addr(a)</code>

<code>set</code>	<code>set</code> <code>var</code> <code>on</code> <code>object</code> <code>to</code> <code>value</code>
<code>set.arg</code>	<code>set.arg</code> <code>n</code> , <code>value</code>
<code>set.break.handler</code>	<code>set.break.handler</code> <code>subroutine.name</code>
<code>set.exit.status</code>	<code>set.exit.status</code> <code>n</code>
<code>set.port.params</code>	<code>set.port.params</code> (<code>a</code> , <code>b</code>)
<code>set.socket.mode</code>	<code>set.socket.mode</code> (<code>a</code> , <code>b</code> , <code>c</code>)
<code>setleft</code>	<code>setleft</code> <code>index</code> <code>from</code> <code>file.var</code> — <code>no</code> <code>then</code> / <code>else</code>
<code>setnls</code>	<code>setnls</code> <code>key</code> , <code>value</code>
<code>setpu</code>	<code>setpu</code> <code>on</code> <code>unit</code> , <code>key</code> , <code>value</code>
<code>setrem</code>	<code>setrem</code> <code>position</code> <code>on</code> <code>var</code>
<code>setright</code>	<code>setright</code> <code>index</code> <code>from</code> <code>file.var</code> — <code>no</code> <code>then</code> / <code>else</code>
<code>setting</code>	clause of <code>find</code> , <code>findstr</code> , <code>locate</code> , <code>remove</code> , <code>selectleft</code> , <code>selectright</code> (<i>clause word</i>)
<code>shift</code>	<code>shift</code> (<code>a</code> , <code>b</code>)
<code>sin</code>	<code>sin</code> (<code>a</code>)
<code>sleep</code>	<code>sleep</code> { <code>seconds</code> } · <code>sleep</code> <code>hh:mm:ss</code>
<code>socket.info</code>	<code>socket.info</code> (<code>a</code> , <code>b</code>)
<code>soundex</code>	<code>soundex</code> (<code>a</code>)
<code>soundexs</code>	<code>soundexs</code> (<code>a</code>)
<code>space</code>	<code>space</code> (<code>a</code>)
<code>spaces</code>	<code>spaces</code> (<code>a</code>)
<code>splice</code>	<code>splice</code> (<code>a</code> , <code>b</code> , <code>c</code>)
<code>sqrt</code>	<code>sqrt</code> (<code>a</code>)
<code>squote</code>	<code>squote</code> (<code>a</code>)
<code>sselect</code>	<code>sselect</code> { <code>file.var</code> } { <code>by</code> <code>field</code> } { <code>to</code> <code>list.no</code> }

status	<code>status()</code>
step	clause of <code>for</code> : <code>for</code> $v = a$ <code>to</code> b <code>step</code> n (clause word)
stop	<code>stop</code> { <i>message</i> }
str	<code>str</code> (a, b)
strs	<code>strs</code> (a, b)
sub	<code>sub</code> <i>name</i> ($arg, \dots$) — synonym of <code>subroutine</code>
subr	<code>subr</code> (" <i>name</i> " { $arg, \dots$ })
subroutine	<code>subroutine</code> <i>name</i> ($arg, \dots$)
substitute	<code>substitute</code> (<i>arr, from, to, delimiter</i>)
substrings	<code>substrings</code> (a, b, c)
sum	<code>sum</code> (a)
summation	<code>summation</code> (a)
swap	<code>swap</code> (<i>string, from, to</i> { <i>occurrences</i> { <i>start</i> } })
swapcase	<code>swapcase</code> (a)
sysmsg	<code>sysmsg</code> (<i>number</i> { <i>substitution, \dots</i> })
system	<code>system</code> (a)

T

tan	<code>tan</code> (a)
tclread	<code>tclread</code> <i>var</i>
terminfo	<code>terminfo</code> (<i>capability</i> { $arg, \dots$ })
then	clause of <code>if</code> , and of every statement that has an <code>else</code> (clause word)
time	<code>time</code> ()

timedate	<code>timedate()</code>
timeout	<code>timeout</code> <i>file.var, seconds</i>
to	clause of <code>open</code> , <code>for</code> , <code>equate</code> , <code>readcsv</code> , <code>select</code> , <code>savelist</code> , <code>writcsv</code> (clause word)
trans	<code>trans({dict , } file.name, id, field, action)</code>
transaction	<code>transaction start</code> · <code>transaction commit</code> · <code>transaction abort</code>
trapping	clause of <code>execute</code> — trap the break key (clause word)
trim	<code>trim(string {, character {, option}})</code>
trimb	<code>trimb(a)</code>
trimbs	<code>trimbs(a)</code>
trimf	<code>trimf(a)</code>
trimfs	<code>trimfs(a)</code>
trims	<code>trims(arr {, character {, option}})</code>

U

umask	<code>umask(a)</code>
unassigned	<code>unassigned(a)</code>
unlock	<code>unlock {n}</code>
until	clause of <code>loop</code> : <code>loop</code> ... <code>until</code> <i>condition</i> ... <code>repeat</code>
upcase	<code>upcase(a)</code>

V

vartype	<code>vartype(a)</code>
----------------	---------------------------

<code>void</code>	<code>void</code> <i>function.call</i> — call a function and discard its value
-------------------	--

<code>vslice</code>	<code>vslice(<i>a</i>, <i>b</i>)</code>
---------------------	---

W

<code>waiting</code>	clause of <code>input</code> : <code>input</code> <code>var</code> <code>waiting</code> <i>seconds</i> (clause word)
----------------------	---

<code>wake</code>	<code>wake</code> <i>user.no</i>
-------------------	----------------------------------

<code>weofseq</code>	<code>weofseq</code> <i>file.var</i> { <code>on error</code> ... }
----------------------	--

<code>while</code>	clause of <code>loop</code> : <code>loop</code> ... <code>while</code> <i>condition</i> ... <code>repeat</code>
--------------------	---

<code>write</code>	<code>write</code> <i>var</i> <code>to</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... } — inside a transaction the lock must already be held
--------------------	---

<code>write.socket</code>	<code>write.socket(<i>a</i>, <i>b</i>, <i>c</i>, <i>d</i>)</code>
---------------------------	---

<code>writeblk</code>	<code>writeblk</code> <i>var</i> <code>to</code> <i>file.var</i> { <code>on error</code> ... }
-----------------------	--

<code>writcsv</code>	<code>writcsv</code> <i>value</i> , ... <code>to</code> <i>file.var</i> <code>then</code> ... <code>else</code> ...
----------------------	---

<code>writeseq</code>	<code>writeseq</code> <i>var</i> <code>to</code> <i>file.var</i> { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
-----------------------	--

<code>writeseqf</code>	<code>writeseqf</code> <i>var</i> <code>to</code> <i>file.var</i> { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
------------------------	---

<code>writeu</code>	<code>writeu</code> <i>var</i> <code>to</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... }
---------------------	--

<code>writew</code>	<code>writew</code> <i>var</i> <code>to</code> <i>file.var</i> , <i>id</i> , <i>field</i> { <code>on error</code> ... }
---------------------	---

<code>writevu</code>	<code>writevu</code> <i>var</i> <code>to</code> <i>file.var</i> , <i>id</i> , <i>field</i> { <code>on error</code> ... }
----------------------	--

X

<code>xlate</code>	<code>xlate({<i>dict</i>, } <i>file.name</i>, <i>id</i>, <i>field</i>, <i>action</i>)</code>
--------------------	--

<code>xtd</code>	<code>xtd(<i>a</i>)</code>
------------------	------------------------------

See also

[SD Basic - Program Structure](#) · [SD Basic - Program Control](#) · [SD Basic - File Handling](#) · [SD Basic - System and Environment](#).